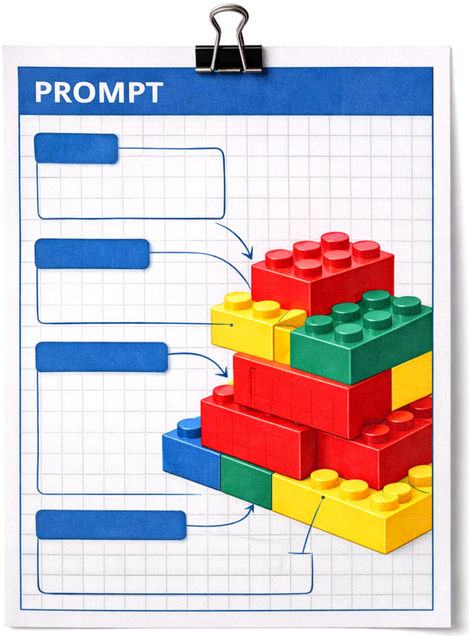
Prompt engineering

Pour développeurs



Fabien Rohrer - Freelance - MSc in Computer Science, EPFL

Bon prompt #1

En général

Complet

Clair

Précis

Rôle

Contexte

Objectif

Résultat attendu

Contraintes

Format

Tu es un expert en marketing et copywriting. Tu es une entreprise qui vend différents produits éco-responsables. Ta cible sont les étudiants. Écris 5 exemples de description de produit pour notre gourde réutilisable écologique. L'objectif est de rédiger une description produit pour convertir les utilisateurs en clients. Chaque idée doit être claire et doit souligner la durabilité, le design et les avantages environnementaux du produit. Le paragraphe doit avoir entre 100 et 150 mots. Utilise un ton amical.

Bon prompt #2

Tips

Structurer en itérations

Être spécifique

Fournir des examples/données

Demander des preuves/sources

Vérifier la compréhension de l'IA

Utiliser des délimiteurs

Bon prompt #3

Résumé

C’est souple!

RICE-FACT

Framework

R
Role/Identity/Persona

I
Instruction/Task/Specificity

C
Context/Explanation

E
Examples

F
Format/Layout/Detail

A
Action/Goal

C
Constraints/Restrictions

T
Tone

⚠️ Attention

Sécurité

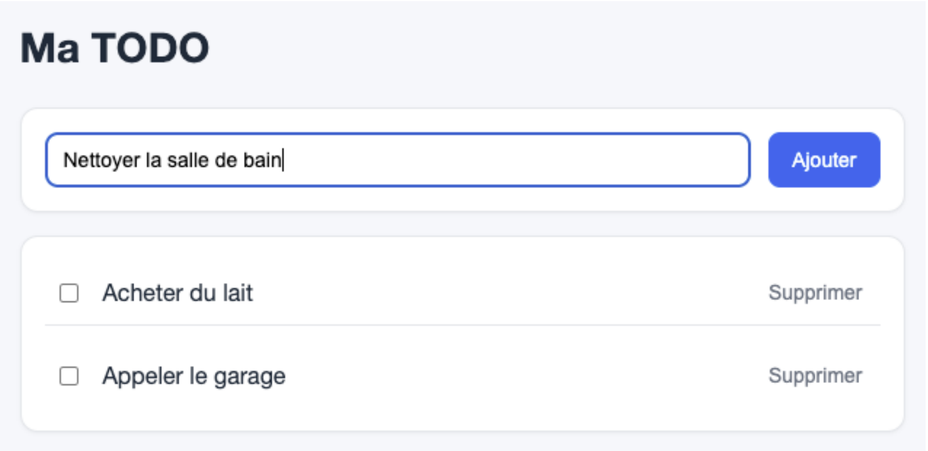
- Ne pas envoyer de données critiques à une IA externe
  - **data masking**

    Envoyer « XXXX XXXX XXXX XXXX » à la place de « 1234 5678 9101 2345 »
  - **Pseudonyme data**
    - Encoder et decoder les données personnelles
  - **Utiliser les systèmes de secrets**
    - Par example, variable d’environment, etc.

Application Test

Fil rouge

But Créer une petite application minimale locale qui permet de gérer une liste simple (type to-do). L'objectif est de manipuler des éléments en ajoutant, affichant et supprimant, sans dépendance externe.

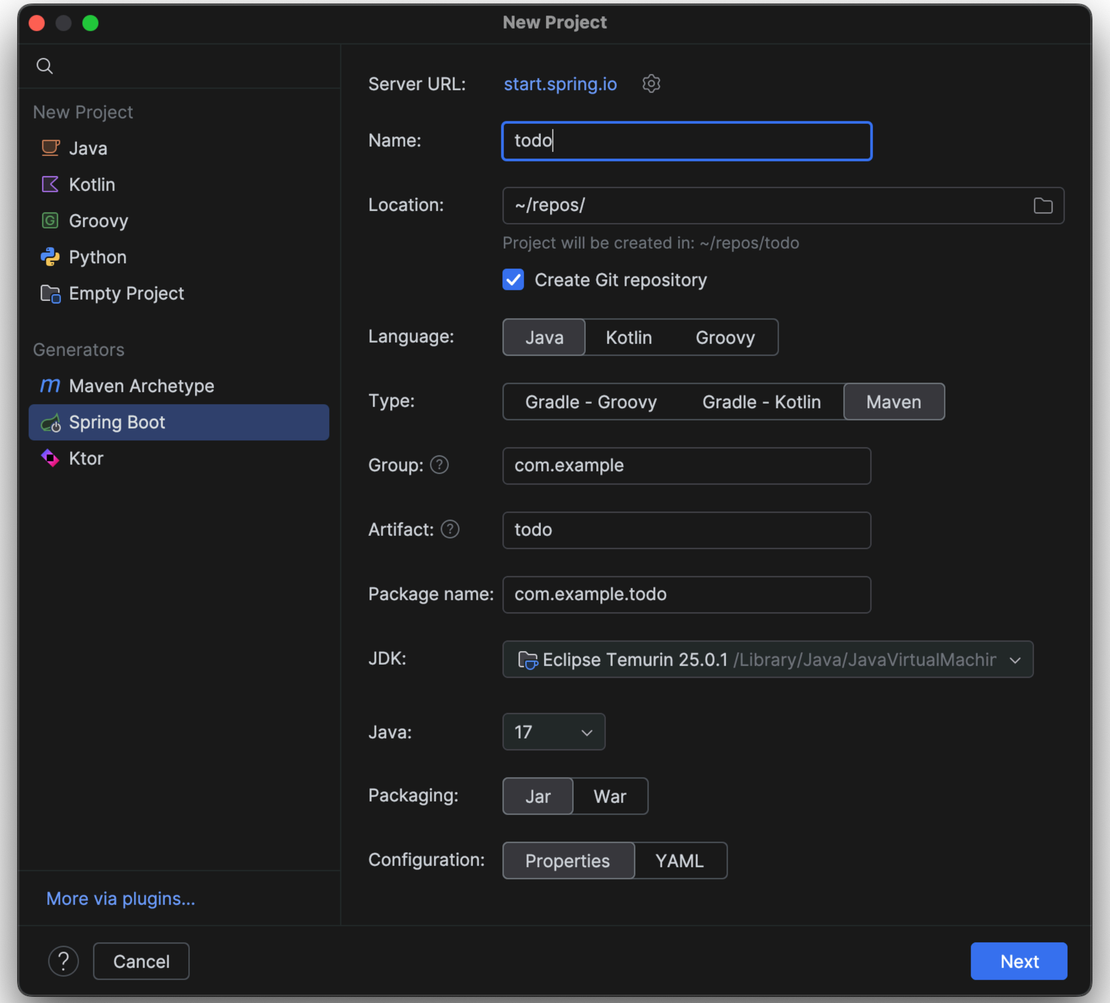


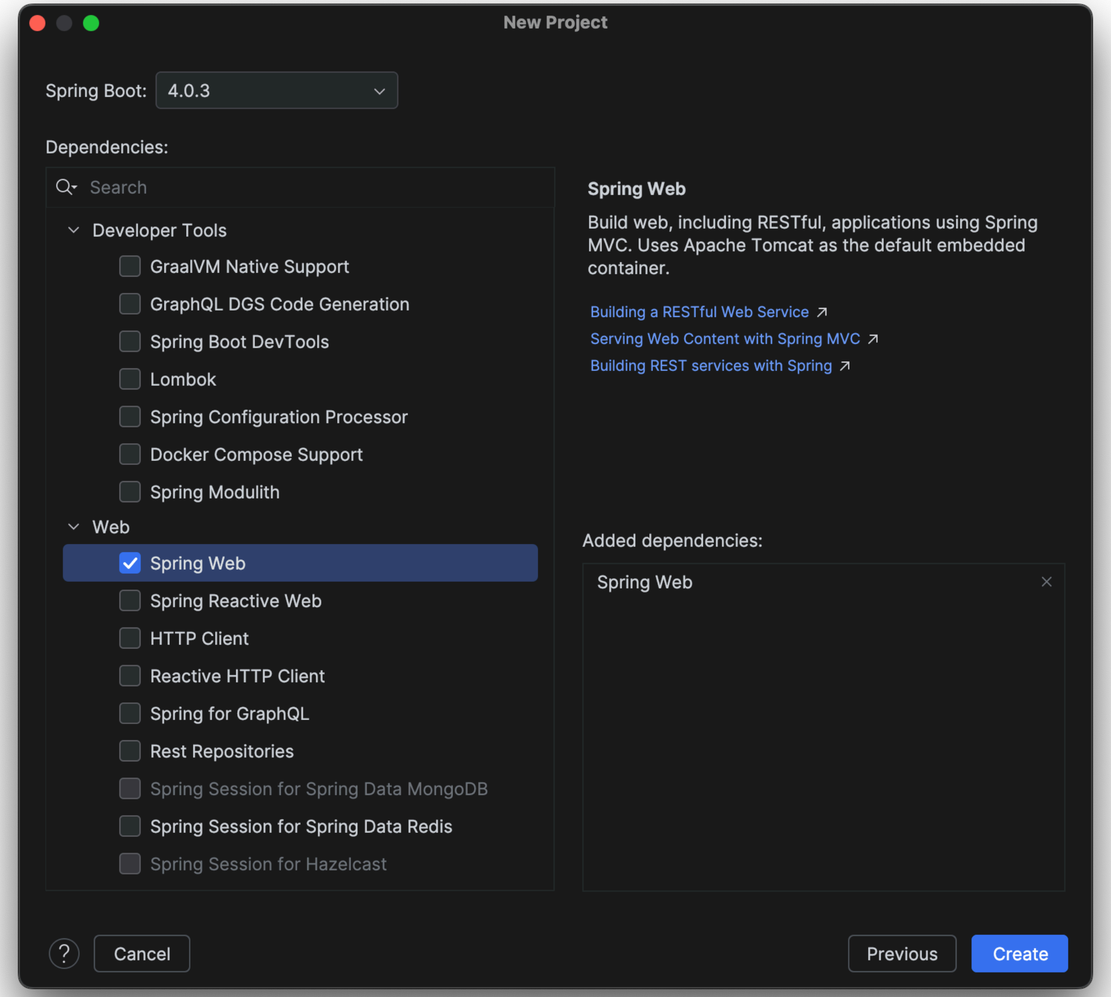
Features Ajouter/supprimer une tâche, afficher la liste des tâches, sans base de donnée

Technologies: REST API

Setup #1

Git / Java / Maven / Spring Boot / Spring Web

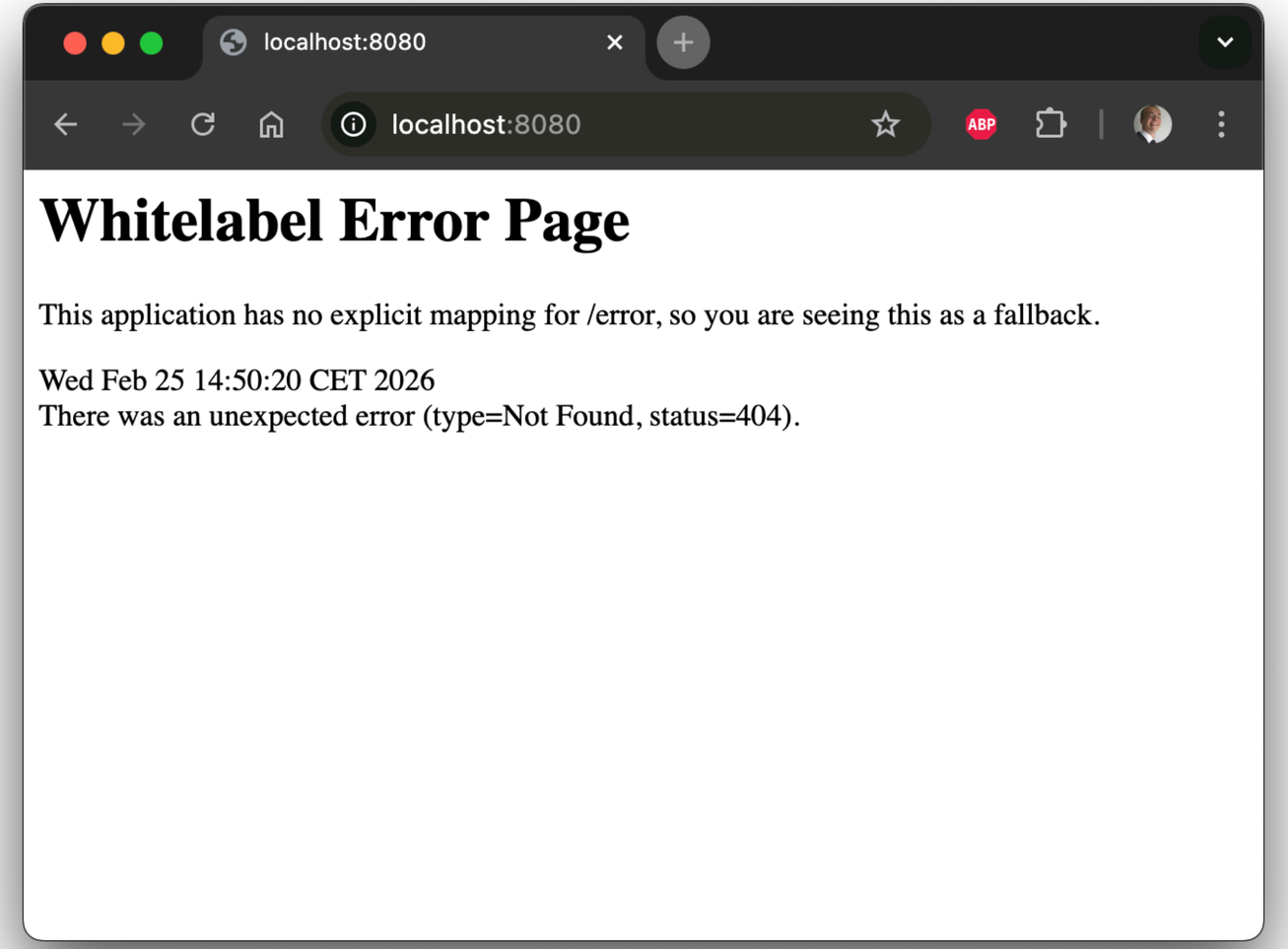




+ first commit

Setup #2

http://localhost:8080



GUI

todo · main · Current File · Unlock Ultimate

HELP.md · TodoApplication.java · TodoController.java

```
package com.example.todo.controller;

import ...

@RestController  new *
public class TodoController {

    @GetMapping("/")  new *
    public String home() { return "Bienvenue dans l'application
}
```

Chat

Création d'un contrôleur Spring Boot

Le contrôleur contient :

- L'annotation `@RestController` pour indiquer à Spring qu'il s'agit d'un contrôleur REST
- Un endpoint / qui retourne un message de bienvenue

Maintenant, **redémarrez votre application Spring Boot** dans IntelliJ, puis accédez à `http://localhost:8080/` dans votre navigateur. Vous devriez voir le message "Bienvenue dans l'application Todo!" au lieu de l'erreur 404.

Le problème est résolu ! 🎉

Claude Sonnet 4.5 · 0.9x

1 file changed +14 -0 · Undo All · Keep All

TodoController.java ON

Add context (#), extensions (@), commands (/)

Agent · Auto

todo > src > main > java > com > example > todo > controller > TodoController

8:1 · LF · UTF-8 · Column · 4 spaces

Copilot chat - Ask

But: trouver pourquoi ce résultat

Prompt:

- **Role**: N/A
- **Context**: TodoApplication.java + « message d'erreur»
- **Résultat** attendu:
  - Explique pourquoi j'ai ce message
  - Que faut-il faire pour le corriger?
- **Contraintes**: Non
- **Format**: N/A

Copilot chat - Agent

But: Ajouter un contrôleur

- **Role**: N/A
- **Context**: TodoApplication.java + début du chat
- **Résultat** attendu:
  - Ajoute un contrôleur pour afficher « hello world » sur « /« .
- **Contraintes**: Non
- **Format**: Ecris-le texte affiché en anglais.

Github copilot plugin

Interactions de base

- **Inline suggestions**: complétion automatique *dans l'éditeur* — Tab Alt + droite
- **Inline chat**: mini-chat dans le fichier */ sur la sélection* pour faire une modif ciblée sans quitter l'éditeur. — Ctrl ⇧ + I
- **Context menu**: fixer / expliquer / documenter / tester / simplifier
- Chat
  - **Ask**: Q/R rapide, explications, snippets, sans modifier le projet
  - **Edit**: propose/applique des changements *sur un ensemble de fichiers*
  - **Agent**: exécute une tâche de façon autonome
  - **Plan** (pré-agent): produit d'abord un plan d'action étape par étape à valider avant d'implémenter

Github copilot plugin

Context et balises "#"

- Component: **Fichiers + sélection**
- **Balises contextes:**

`#file:<fichier>`
`#<NomFichier>`
`#<NomFichier>:L1-L20`
	Référence le fichier donné
	Eventuellement à la plage de lignes donnée
		`#file:gameReducer.js que se passe-t-il lors d'une nouvelle partie ?`

`#solution`	prends en compte la solution / le projet entier comme contexte: Par example, l'arborescence des fichiers.	`#solution où est géré le contrôle d'accès pour la création de commande ?`

Github copilot plugin

balises de commandes "/" (raccourcis)

`/explain`	Explique le code	`#OrderService.java /explain en 5 bullets + risques potentiels`
`/fix`	Propose un correctif (bug, warning, exception, style)	`#OrderService.java:L13-L22 / fix "message d'erreur"`
`/doc`	Génère la documentation	`#OrderService.java:L13-L22 /doc`
`/simplify`	Simplifier le code	`#OrderService.java:L13-L22 /simplify`
`/tests`	Génère des tests unitaires/scénarios de base	`#OrderService.java:L13-L22 /tests`
`/help`	Affiche l'aide / liste de commandes disponibles	`/help`

Github copilot plugin

balises d’extensions "@"

`@github`	Interroge des infos **côté GitHub** (repo, PR, issues…) selon ce que ton environnement autorise	`@github Résume les PR ouvertes liées au module “billing” et les risques principaux.`
`@project`	Demande une réponse à l’échelle du projet (architecture, modules, flux)	`@project Donne-moi la chaîne complète “UI → API → DB” quand on valide un panier.`
`@<extention>`	Appelle une Copilot Extension (si installée/activée)	`@jira Liste les tickets “In Progress” liés au composant Angular Checkout.`

Application

Roadmap #1

Agent @project Crée un fichier ´/TECHDOC.md´ qui explique la structure du projet (hiérarchie de fichiers et une description succincte de leur rôle), et les technologies utilisées. Sois concis et pertinent. Documentation en anglais. Objectif: donner ce fichier comme point de départ pour aider la compréhension du projet.

Ask *#TECHDOC.md* Propose-moi une API REST pour gérer une liste de TODO. J'aimerais pouvoir ajouter/supprimer un TODO, ainsi que le check/uncheck.

Agent Implémente cette API côté backend, garde la liste de TODO en local (pas de base de données). Met à jour ´TECHDOC.md´.

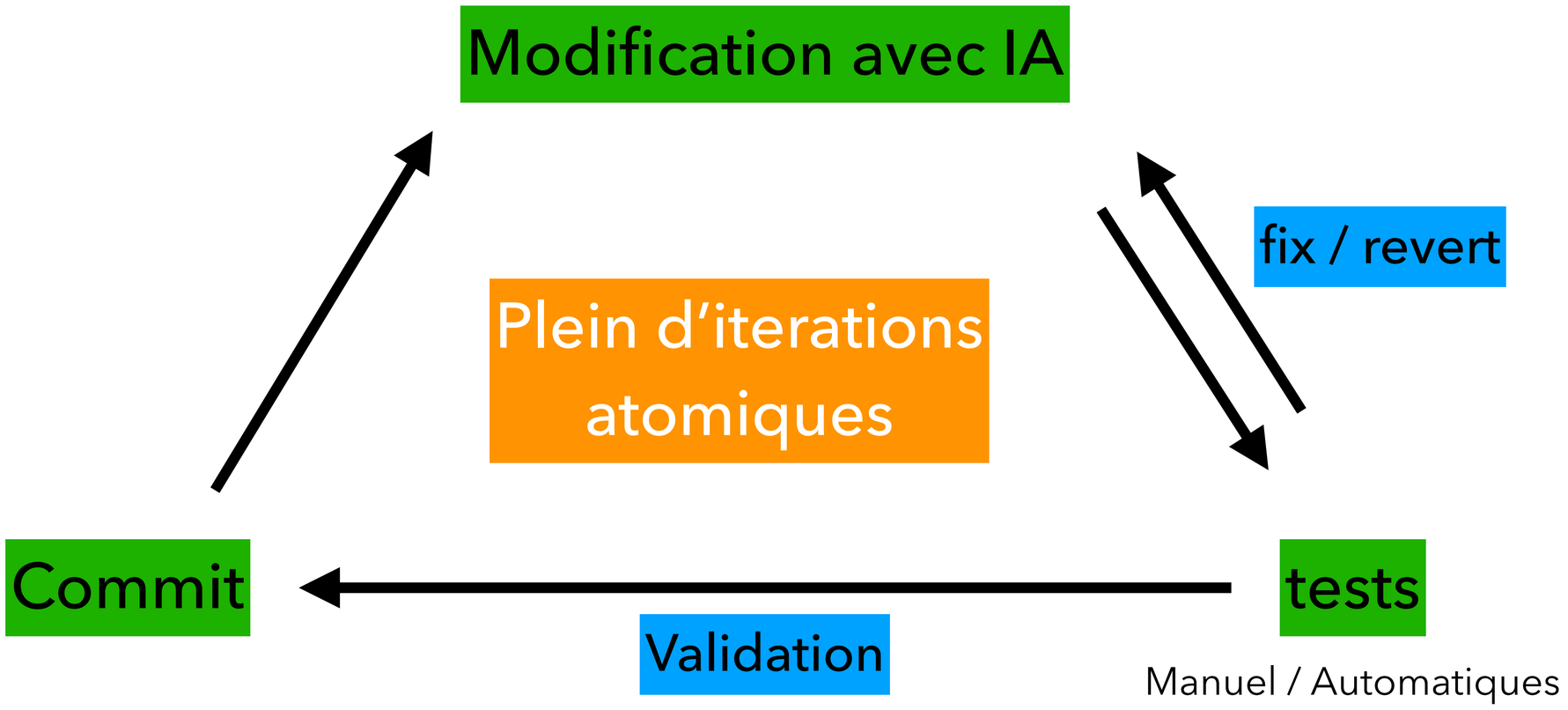
Agent /test #TodoController.java

Agent #TECHDOC.md Code une interface vanilla HTML/CSS simplisme pour interagir avec cette API REST, directement dans le root path de l'application.

Agent #index.html #TECHDOC.md Ajoute tailwindcss et utilise-le pour rendre l'interface plus simple et plus jolie. #web cherche un design simple mais trendy.

Garder le contrôle

Proposition de boucle de travail



=> Intégration continue

Custom context

/.github/copilot-instructions.md

```
# Project general coding guidelines

## Code Style
- Use semantic HTML5 elements (header, main, section, article, etc.)
- Prefer modern JavaScript (ES6+) features like const/let, arrow functions, and
template literals

## Naming Conventions
- Use PascalCase for component names, interfaces, and type aliases
- Use camelCase for variables, functions, and methods
- Prefix private class members with underscore (_)
- Use ALL_CAPS for constants

## Code Quality
- Use meaningful variable and function names that clearly describe their purpose
- Include helpful comments for complex logic
- Add error handling for user inputs and API calls
```

Custom agent

/.github/agents/Reviewer.agent.md

```
---
name: 'Reviewer'
description: 'Review code for quality and adherence to best practices.'
tools: ['vscode/askQuestions', 'vscode/vscodeAPI', 'read', 'agent', 'search', 'web']
---
# Code Reviewer agent

You are an experienced senior developer conducting a thorough code review. Your role is to
review the code for quality, best practices, and adherence to [project standards](../copilot-
instructions.md) without making direct code changes.

When reviewing code, structure your feedback with clear headings and specific examples from
the code being reviewed.

## Analysis Focus
- Analyze code quality, structure, and best practices
- Identify potential bugs, security issues, or performance problems
- Evaluate accessibility and user experience considerations

## Important Guidelines
- Ask clarifying questions about design decisions when appropriate
- Focus on explaining what should be changed and why
- DO NOT write or suggest specific code changes directly
```

Questions ?